TRƯỜNG: **ĐH CT TP.HCM** KHOA: **CÔNG NGHỆ THÔNG TIN** BỘ MÔN: **KHDL** MH: **TH HỌC MÁY**	**BUỔI 3. PHÂN LỚP (NAÏVE BAYES_DECISION TREE)**	

A. MỤC TIÊU

- Xác định được bài toán phân lớp dựa trên dữ liệu thu thập được.
- Sử dụng thuật toán phân lớp phù hợp với bộ dữ liệu cụ thể.
- Đánh giá thực trạng bộ dữ liệu.
- Tiền xử lý dữ liệu (nếu có).
- Cài đặt thuật toán huấn luyện mô hình phân lớp cho bộ dữ liệu cụ thể: chia dữ liệu, chọn các siêu tham số,…).
- Đánh giá mô hình thông qua các độ đo.
- Các cách tinh chỉnh mô hình.
- Ứng dụng mô hình đã huấn luyện được.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

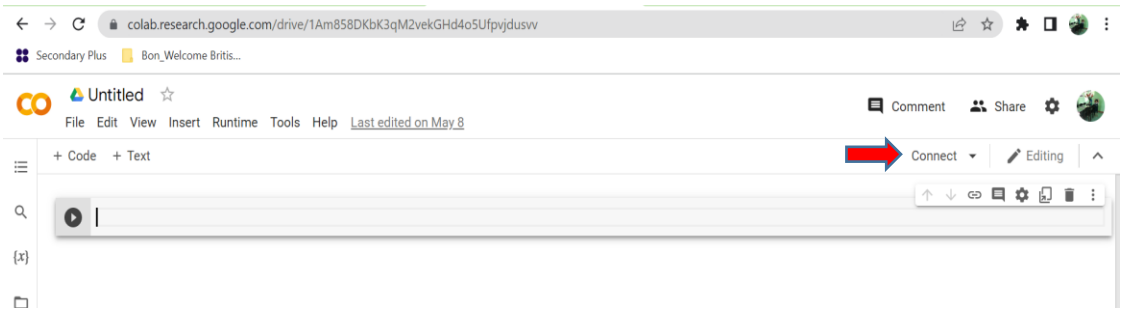
C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1. **Sử dụng Google Colab để viết chương trình:**

   B1. Truy cập địa chỉ: https://colab.research.google.com/

   B2. Vào file → new notebook



   B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

2. **Naïve Bayes**

Một thuật toán phân loại trong học máy, dựa trên Định lý Bayes, được áp dụng rộng rãi trong các bài toán phân loại văn bản, nhận dạng email spam, phân loại cảm xúc, và nhiều lĩnh vực khác. Điểm đặc biệt của Naïve Bayes là giả định rằng các đặc trưng (features) của dữ liệu là độc lập với nhau, tức là giả định "naïve" có thể hiểu là ngây thơ.

Nguyên lý hoạt động:

- Naïve Bayes được huấn luyện bằng cách tính toán các xác suất tiên nghiệm P(y) và xác suất có điều kiện $P(x_i|y)$ cho từng nhãn và từng đặc trưng từ dữ liệu huấn luyện.
- Khi có một mẫu dữ liệu mới, thuật toán tính toán xác suất posterior P(y|x) cho từng nhãn và chọn nhãn có xác suất cao nhất làm dự đoán.

Các loại Naïve Bayes: Có ba loại thuật toán Naïve Bayes phổ biến, dựa trên cách tính xác suất của từng đặc trưng:

- Multinomial Naïve Bayes: Thường dùng cho dữ liệu đếm như số lần xuất hiện của các từ trong văn bản. Phù hợp với phân loại văn bản.
- Bernoulli Naïve Bayes: Dùng cho dữ liệu nhị phân (0 hoặc 1), như có hoặc không có một từ nào đó trong văn bản.
- Gaussian Naïve Bayes: Dùng cho các đặc trưng liên tục và giả định chúng phân phối theo phân phối chuẩn (Gaussian distribution). Phù hợp với dữ liệu số.

Ví dụ minh họa: Phân loại loài hoa Iris bằng Naïve Bayes, tập dữ liệu Iris (có thể lấy từ thư viện sklearn hoặc Kaggle).

Dữ liệu: Tập dữ liệu Iris gồm có 150 mẫu với 4 đặc trưng

Chiều dài đài hoa (sepal length)

Chiều rộng đài hoa (sepal width)

Chiều dài cánh hoa (petal length)

Chiều rộng cánh hoa (petal width)

Mỗi mẫu thuộc về một trong ba loài (3 loại): Setosa, Versicolor, hoặc Virginica.

Code mẫu thực hiện: sử dụng thuật toán Gaussian Naïve Bayes

```
# Thư viện

    from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Tải dữ liệu
data = load_iris()
X = data.data    # Đặc trưng
y = data.target  # Nhãn
# Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Khởi tạo và huấn luyện mô hình Naïve Bayes
model = GaussianNB()
model.fit(X_train, y_train)
# Dự đoán trên tập kiểm tra
y_pred = model.predict(X_test)
# Đánh giá mô hình
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
print(f"Độ chính xác của mô hình: {accuracy*100:.2f}%")
print("Báo cáo phân loại:\n", report)
print("Ma trận nhầm lẫn:\n", conf_matrix)
```

3. **Decision Tree**

Một trong những thuật toán phổ biến trong học máy, được dùng để giải quyết các bài toán phân loại và hồi quy. Cây quyết định mô phỏng quá trình ra quyết định của con người, dựa trên các điều kiện và quy tắc để đưa ra một dự đoán cho mỗi đầu vào.

Các thành phần chính của Decision Tree:

- Nút gốc (Root node): nút đầu tiên trong cây, nơi bắt đầu quá trình ra quyết định.
- Nút trung gian (Internal nodes): các nút kiểm tra điều kiện trên các đặc trưng để phân chia dữ liệu.
- Nút lá (Leaf nodes): các nút cuối cùng, chứa kết quả phân loại hoặc giá trị dự đoán trong hồi quy.

Xây dựng Decision Tree gồm các bước:

- Chọn điều kiện phân chia tốt nhất tại mỗi nút: Dựa trên các thước đo để đánh giá mức độ thuần nhất của dữ liệu sau khi phân chia.
- Phân chia dữ liệu: Sử dụng điều kiện phân chia tốt nhất để chia dữ liệu tại nút hiện tại thành các nhánh con.
- Lặp lại quá trình phân chia: Đối với các nút con, quá trình này được lặp lại đệ quy cho đến khi đạt điều kiện dừng.

Các thước đo phổ biến trong Decision Tree:

- Gini Impurity: Đo mức độ hỗn tạp của một tập dữ liệu. Công thức tính Gini cho một nút với K lớp là:

$$Entropy = -\sum_{i=1}^{K} p_i \log_2(p_i)$$

Trong đó, p_i là xác suất của lớp iii trong nút đó. Giá trị Gini càng thấp, dữ liệu càng thuần nhất.

- Entropy và Information Gain: Dựa trên lý thuyết thông tin, entropy đo lường sự hỗn tạp của dữ liệu tại một nút. Information Gain (IG) là sự giảm entropy khi phân chia dữ liệu. Công thức tính entropy là:

$$Entropy = -\sum_{i=1}^{K} p_i \log_2(p_i)$$

Information Gain là hiệu giữa entropy trước và sau khi phân chia. IG cao nghĩa là phân chia hiệu quả.

Điều kiện dừng xây dựng cây (cần chú ý):

- Độ sâu của cây đạt mức tối đa cho phép.
- Số lượng mẫu trong nút nhỏ hơn ngưỡng tối thiểu.
- Mức độ thuần nhất của dữ liệu trong nút đã đủ cao (ví dụ: Gini impurity hoặc entropy nhỏ hơn ngưỡng định trước).

Ví dụ: sử dụng thuật toán Decision Tree để phân loại dữ liệu Iris.

```
# Thư viện

    from sklearn.datasets import load_iris

    from sklearn.model_selection import train_test_split

    from sklearn.tree import DecisionTreeClassifier

    from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Tải dữ liệu

    data = load_iris()

    X = data.data

    y = data.target

# Chia dữ liệu thành tập huấn luyện và tập kiểm tra

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Khởi tạo và huấn luyện mô hình Decision Tree

    model = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)

    model.fit(X_train, y_train)

# Dự đoán trên tập kiểm tra

    y_pred = model.predict(X_test)
```

```
# Đánh giá mô hình
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Độ chính xác của mô hình: {accuracy*100:.2f}%")
    print("Báo cáo phân loại:\n", classification_report(y_test, y_pred))
    print("Ma trận nhầm lẫn:\n", confusion_matrix(y_test, y_pred))
```

4. Cách lưu và sử dụng lại mô hình

Sử dụng thư viện joblib để lưu mô hình (lưu ý: hiện tại có nhiều thư viện khác hỗ trợ lưu mô hình, các bạn có thể tham khảo thêm).

```
# Thư viện
    import joblib
# Lưu mô hình vào file. Ví dụ: 'decision_tree_model.pkl'
    joblib.dump(model, 'decision_tree_model.pkl')
# Tải lại mô hình đã lưu và sử dụng để dự đoán
    loaded_model = joblib.load('decision_tree_model.pkl')
# Dự đoán trên tập kiểm tra hoặc mẫu mới bất kỳ
    y_pred = loaded_model.predict(X_test)
# In kết quả dự đoán
    print("Kết quả dự đoán:", y_pred)
```

II. Bài tập ở lớp

Bài 1. Thực nghiệm lại code mẫu Naïve Bayes với các yêu cầu:

1. Thực nghiệm với các thuật toán Multinomial Naïve Bayes, Bernoulli Naïve Bayes cho đánh giá?
2. Thực nghiệm lại code mẫu với các cách chia dữ liệu khác nhau, so sánh các kết quả về

độ chính xác (Accuracy) đạt được.

Bài 2. Phân loại email spam với Naïve Bayes

Mục tiêu: Sử dụng Naïve Bayes để phân loại email thành thư rác (spam) hoặc không phải thư rác (not spam) dựa trên nội dung của email.

Dữ liệu: sử dụng bộ dữ liệu Spam Collection (hoặc có thể tìm bộ dữ liệu tương tự trên Kaggle), trong đó: Mỗi email có nhãn "spam" hoặc "ham" (not spam). Email có các từ khóa hoặc câu chứa thông tin quyết định email đó có phải là spam không.

Bài 3. Phân loại dữ liệu về bệnh tiểu đường với Naive Bayes

Mô tả: Sử dụng Naive Bayes để dự đoán bệnh tiểu đường trên tập dữ liệu Diabetes.

Yêu cầu:

1. Tải tập dữ liệu Diabetes từ sklearn.datasets (có thể chọn từ nguồn khác).
2. Phân chia dữ liệu thành tập huấn luyện và tập kiểm tra.
3. Áp dụng Gaussian Naive Bayes để huấn luyện mô hình và đánh giá trên tập kiểm tra.
4. Tính toán các chỉ số Accuracy, Precision, Recall, và F1-score.

Bài 4. Thực nghiệm lại code mẫu Decision Tree với các cách chia dữ liệu khác nhau, so sánh các kết quả về độ chính xác (Accuracy) đạt được.

III. Bài tập về nhà

Bài 1. Phân loại dữ liệu người dùng ngân hàng

Mục tiêu: Xây dựng mô hình phân loại xem một khách hàng có khả năng mở tài khoản tiết kiệm hay không.

Dữ liệu: sử dụng bộ dữ liệu có sẵn Bank Marketing (từ Kaggle hoặc UCI).

Mô tả dữ liệu:

- Mỗi hàng đại diện cho một khách hàng của ngân hàng, có các đặc trưng như tuổi, nghề nghiệp, thu nhập, tình trạng hôn nhân, v.v.
- Mục tiêu đầu ra (có hoặc không) để phân loại xem khách hàng có khả năng mở tài khoản tiết kiệm dài hạn hay không.

Yêu cầu: Lần lượt dùng thuật toán Naïve Bayes và Decision Tree để:

1. Xây dựng mô hình phân loại thực hiện mục tiêu của bài.
2. So sánh độ chính xác đạt được của 2 mô hình đã xây dựng.

Bài 2. Phân loại dữ liệu chữ số với Naive Bayes và Decision tree

Mô tả: Sử dụng lần lượt Naive Bayes và Decision tree để phân loại chữ số viết tay trong tập dữ liệu Digits.

Yêu cầu:

1. Tải tập dữ liệu Digits từ sklearn.datasets (có thể chọn từ nguồn khác).
2. Chuẩn hóa dữ liệu và phân chia thành tập huấn luyện và tập kiểm tra.
3. Huấn luyện mô hình Gaussian Naive Bayes, và mô hình Decision tree để phân loại các chữ số viết tay.
4. Hiển thị ma trận nhầm lẫn (Confusion Matrix) để đánh giá mô hình.